

ReachInbox Email Scheduler: Complete Project Handoff

ReachInbox Email Scheduler: Complete Project Handoff

Build and run a production-style full-stack email scheduling application inspired by ReachInbox.

Assignment Goal

The application must allow an authenticated user to:

1. Login using real Google OAuth.
2. Upload a CSV or text file containing email addresses.
3. Enter an email subject and body.
4. Select a sender.
5. Choose a future start time.
6. Choose a delay between emails.
7. Choose an hourly email limit.
8. Create one separate email job per recipient.
9. Schedule jobs using BullMQ delayed jobs.
10. Send each email individually through SMTP.
11. View scheduled emails.
12. View sent and failed emails.
13. Search emails using Elasticsearch.
14. Monitor jobs using Bull Board.
15. Preserve jobs after backend restart.
16. Prevent the same email from being sent twice.

Do not use cron jobs.

Do not use node-cron, agenda, OS cron, persistent setTimeout, or in-memory scheduling.

Technologies Used

Backend:

- TypeScript
- Node.js
- Express.js
- BullMQ
- Redis
- PostgreSQL
- Prisma ORM
- Nodemailer
- Ethereal SMTP
- Gmail SMTP or another real SMTP provider when real inbox delivery is required
- Elasticsearch
- Passport.js
- Google OAuth 2.0
- Slack OAuth
- Bull Board

- Zod
- JWT authentication
- dotenv

Frontend:

- React
- TypeScript
- Vite
- Axios
- React Router
- CSS/Tailwind-style responsive design
- Lucide icons
- Sonner toast notifications

Infrastructure:

- Docker Desktop
- WSL2/Ubuntu when Docker uses the WSL2 backend
- PostgreSQL Docker container
- Redis Docker container
- Elasticsearch Docker container

Why These Technologies Are Used

PostgreSQL stores users, senders, campaigns, and email jobs permanently.

Prisma provides typed PostgreSQL access and database migrations.

Redis stores BullMQ jobs and distributed hourly rate-limit counters.

BullMQ schedules email jobs for exact future times and keeps delayed jobs after application restarts.

Nodemailer sends individual emails through SMTP.

Ethereal is used for fake SMTP testing and preview URLs required by the assignment.

Gmail, Brevo, SendGrid, Mailgun, or Amazon SES can be used when emails must arrive in real Gmail inboxes.

Elasticsearch indexes email jobs and supports search.

Bull Board provides live BullMQ queue monitoring.

Google OAuth provides real authentication.

Slack OAuth sends notifications when a sender reaches the hourly limit.

React provides the dashboard.

TypeScript provides compile-time type safety.

Project Structure

```
reachinbox-email-scheduler/  
  backend/  
    src/  
      auth/  
      config/  
      middleware/  
      queues/  
      routes/
```

```
services/
types/
utils/
workers/
app.ts
server.ts
prisma/
  schema.prisma
  migrations/
  seed.ts
tests/
  csv.test.ts
.env
.env.example
package.json
tsconfig.json

frontend/
src/
  services/
  App.tsx
  main.tsx
  styles.css
  types.ts
  vite-env.d.ts
.env
.env.example
package.json
vite.config.ts

docker-compose.yml
sample-leads.csv
README.md
package.json
.gitignore
```

Database Models

User

- id
- googleId
- name
- email
- avatar
- createdAt
- updatedAt

Sender

- id
- userId
- email
- name
- SMTP username
- SMTP password
- createdAt

EmailCampaign

- id
- userId
- senderId
- subject
- body

- startTime
- delayBetweenEmails
- hourlyLimit
- totalRecipients
- createdAt
- updatedAt

EmailJob

- id
- campaignId
- senderId
- recipientEmail
- subject
- body
- scheduledAt
- sentAt
- status
- bullJobId
- attempts
- errorMessage
- messageId
- createdAt
- updatedAt

Email statuses: scheduled, processing, sent, failed, delayed

SlackConnection

- id
- userId
- accessToken
- teamId
- teamName
- connected
- createdAt
- updatedAt

SMTP passwords and Slack tokens must never be returned to the frontend.

Windows Installation Requirements

Install:

- Node.js LTS
- npm
- Docker Desktop
- WSL2 with Ubuntu if Docker is configured for WSL2
- Git, optional
- Modern browser

Recommended minimum:

- Node.js 20 or newer
- 8 GB RAM
- At least 10 GB free storage
- Internet connection

Ubuntu is only used by Docker's WSL2 backend. Application code can still be run from PowerShell and VS Code.

Verify Windows Tools

Open a new PowerShell terminal:

```
node --version
npm.cmd --version
docker --version
docker compose version
wsl.exe --status
```

If commands are not recognized, refresh PATH:

```
$env:Path += ";C:\Program Files\nodejs;C:\Program
Files\Docker\Docker\resources\bin"
```

If PowerShell blocks npm.ps1, always use npm.cmd instead of npm.

Docker Desktop must be open and show: **Engine running**

Start Docker Infrastructure

Go to the project folder:

```
cd C:\Users\manoj\Desktop\outbox
```

Start PostgreSQL, Redis, and Elasticsearch:

```
docker compose up -d
```

Check containers:

```
docker compose ps
```

Expected services:

- PostgreSQL: localhost:5432
- Redis: localhost:6379
- Elasticsearch: localhost:9200

All containers should show Running or Healthy.

Docker uses persistent volumes: postgres_data, redis_data, elasticsearch_data. These volumes preserve data after container restarts.

Install Node Dependencies

From the project root:

```
npm.cmd install
npm.cmd --prefix backend install
npm.cmd --prefix frontend install
```

Create environment files:

```
Copy-Item backend\.env.example backend\.env
Copy-Item frontend\.env.example frontend\.env
```

Never commit backend/.env or frontend/.env.

Backend Environment File

Open:

```
notepad C:\Users\manoj\Desktop\outbox\backend\.env
```

Configure the database and infrastructure:

```
PORT=4000
```

```
DATABASE_URL=postgresql://reachinbox:reachinbox@localhost:5432/reachinbox
```

```
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=
ELASTICSEARCH_URL=http://localhost:9200
FRONTEND_URL=http://localhost:5173
JWT_SECRET=create-a-long-random-secret
WORKER_CONCURRENCY=5
MAX_EMAILS_PER_HOUR_PER_SENDER=100
```

Google OAuth Configuration

Create OAuth credentials in Google Cloud Console.

The authorized redirect URI must be exactly:

```
http://localhost:4000/api/auth/google/callback
```

Add these values to backend/.env:

```
GOOGLE_CLIENT_ID=actual-google-client-id
GOOGLE_CLIENT_SECRET=actual-google-client-secret
GOOGLE_CALLBACK_URL=http://localhost:4000/api/auth/google/callback
```

Add the Google account being used for testing as an OAuth test user.

Do not use placeholders. Do not share the Client Secret. The Google Client ID and Google Client Secret are different values.

Slack OAuth Configuration

Slack is optional for the basic email flow but required for rate-limit notification testing.

```
SLACK_CLIENT_ID=actual-slack-client-id
SLACK_CLIENT_SECRET=actual-slack-client-secret
SLACK_REDIRECT_URI=http://localhost:4000/api/slack/callback
```

Add the same callback URL to the Slack application.

The dashboard supports:

- Connect Slack
- Disconnect Slack
- Slack connection status

If Slack is disconnected, rate-limit processing must continue without crashing.

SMTP Configuration for Ethereal

The assignment specifically mentions Ethereal fake SMTP.

```
SMTP_HOST=smtp.ethereal.email
SMTP_PORT=587
SMTP_USER=ethereal-username
SMTP_PASSWORD=ethereal-password
```

Ethereal accepts test messages and displays preview URLs. Ethereal does not deliver messages to Gmail inboxes.

The worker prints a URL similar to:

Ethereal preview: <https://ethereal.email/message/...>

Use that URL to view the test message.

SMTP Configuration for Real Gmail Delivery

If emails must arrive in real Gmail inboxes, use Gmail SMTP:

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-gmail-address@gmail.com
SMTP_PASSWORD=your-gmail-app-password
```

The password must be a Gmail App Password. Do not use:

- Normal Gmail password
- Google OAuth Client ID
- Google OAuth Client Secret

The Gmail account used in SMTP_USER must be the same account that created the App Password. Remove spaces from the App Password.

Other real providers can also be used:

Brevo

```
SMTP_HOST=smtp-relay.brevo.com
SMTP_PORT=587
SMTP_USER=brevo-login-email
SMTP_PASSWORD=brevo-smtp-key
```

SendGrid, Mailgun, and Amazon SES provide their own SMTP host, username, and password.

Prepare Prisma

Run Prisma from the backend directory:

```
cd C:\Users\manoj\Desktop\outbox\backend
```

Generate Prisma client:

```
npm.cmd exec -- prisma generate
```

Create the first migration:

```
npm.cmd exec -- prisma migrate dev --name init
```

If the migration already exists in the copied project:

```
npm.cmd exec -- prisma migrate deploy
```

Return to the root:

```
cd ..
```

The migration creates PostgreSQL tables for users, senders, campaigns, email jobs, and Slack connections.

Start Backend

Open PowerShell terminal 1:

```
cd C:\Users\manoj\Desktop\outbox
npm.cmd --prefix backend run dev
```

Expected output:

API listening on http://localhost:4000

Test the backend:

Invoke-RestMethod http://localhost:4000/health

Expected result: ok: True

Start BullMQ Worker

Open PowerShell terminal 2:

```
cd C:\Users\manoj\Desktop\outbox
npm.cmd --prefix backend run worker
```

Expected output:

Email worker ready

Only one worker should be started initially. The worker uses `WORKER_CONCURRENCY=5`, allowing five email jobs to be processed in parallel.

Start Frontend

Open PowerShell terminal 3:

```
cd C:\Users\manoj\Desktop\outbox
npm.cmd --prefix frontend run dev
```

Expected output:

Local: http://localhost:5173/

Open: http://localhost:5173

Login Flow

1. Open http://localhost:5173.
2. Click Continue with Google.
3. Sign in with a configured Google test user.
4. Google redirects to the backend callback.
5. Backend finds or creates the user.
6. Backend creates a JWT.
7. JWT is stored in an HTTP-only cookie.
8. User is redirected to the dashboard.

The dashboard displays:

- User name
- User email
- User avatar
- Logout button
- Scheduled Emails
- Sent Emails
- Compose New Email
- Slack status

Sender Flow

After login, the frontend calls GET /api/senders. The backend returns the authenticated user's senders. SMTP usernames and passwords are never returned.

If no sender exists, the backend creates a default sender using the configured SMTP credentials. The sender email should match the SMTP account.

CSV Flow

Upload sample-leads.csv or another CSV file:

```
email
john@example.com
sarah@example.com
alex@example.com
```

The parser:

1. Reads the uploaded file.
2. Skips the email header.
3. Trims values.
4. Converts addresses to lowercase.
5. Validates email format.
6. Removes duplicates.
7. Reports invalid addresses.
8. Creates one recipient entry per valid email.

Compose Flow

Click Compose New Email. Enter:

- Sender
- Subject
- Body
- CSV file
- Start time
- Delay between emails
- Hourly limit

Example: Start time 5:00 PM, delay 10 seconds, hourly limit 100.
Click **Schedule emails**.

Individual Email Jobs

The application creates one campaign and one job per recipient. For example:

```
john@example.com -> EmailJob 1
sarah@example.com -> EmailJob 2
alex@example.com -> EmailJob 3
```

Each recipient receives an individual SMTP message. The application never sends all recipients in one To field.

Scheduled times are calculated as:

$$\text{scheduledAt} = \text{startTime} + \text{recipientIndex} * \text{delayBetweenEmails}$$

Example:

Email 1: 5:00:00 PM
Email 2: 5:00:10 PM
Email 3: 5:00:20 PM

BullMQ Job Data

Every BullMQ job contains:

- emailJobId
- campaignId
- recipientEmail
- senderId

Each BullMQ job receives a deterministic ID based on the EmailJob. BullMQ stores delayed jobs in Redis until their scheduled time.

Worker Processing Flow

1. Receives a BullMQ job.
2. Finds the EmailJob in PostgreSQL.
3. Skips the job if it is already sent.
4. Atomically changes scheduled or delayed to processing.
5. Checks the Redis hourly limit.
6. Delays the job if the limit is reached.
7. Sends one email through SMTP.
8. Stores the SMTP message ID.
9. Stores sentAt.
10. Changes the status to sent.
11. Indexes the job in Elasticsearch.

Idempotency

Idempotency prevents duplicate sends. The worker uses an atomic database update:

scheduled -> processing

Only one worker can claim a job. If another worker attempts the same job, it sees that the status is already processing or sent and skips it.

After successful SMTP delivery:

processing -> sent

Already-sent jobs are never sent again during retries or restarts.

Email Status Flow

- Normal flow: scheduled -> processing -> sent
- Hourly rate-limit flow: scheduled -> processing -> delayed -> processing -> sent
- SMTP failure flow: scheduled -> processing -> failed

Status	Meaning
scheduled	Waiting for the BullMQ delay.
processing	Currently owned by a worker.
sent	SMTP accepted the email.
delayed	The hourly rate limit was reached.
failed	SMTP failed after retry handling.

Hourly Rate Limiting

Redis stores distributed counters with key format:

```
email-rate-limit:{senderId}:{hourWindow}
```

If the hourly limit is 3:

- First three emails can send.
- Remaining emails are delayed, not deleted, not permanently failed.
- They can send in the next available hour.

The counter is shared across workers and backend instances. No in-memory rate counter is used.

Slack Notification

When the hourly limit is reached:

1. Redis detects the limit.
2. The worker sends a Slack API message.
3. Redis stores a notification key.
4. Duplicate notifications for the same sender/hour are prevented.
5. If Slack is disconnected, notification is skipped.
6. Email processing continues normally.

Elasticsearch

Email jobs are indexed in an Elasticsearch index named `emails`. Indexed fields include:

- `emailJobId`, `campaignId`, `userId`
- `senderEmail`, `recipientEmail`
- `subject`, `body`, `status`
- `scheduledAt`, `sentAt`, `createdAt`

Search endpoint:

```
GET /api/emails/search?q=john@example.com
```

Search supports recipient email, subject, body, and status. Results are filtered by authenticated user ID.

Dashboard Scheduled Page

GET `/api/emails/scheduled` — table shows Recipient, Subject, Scheduled time, Status. Includes loading state, empty state, search, refresh button, status badges.

Dashboard Sent Page

GET `/api/emails/sent` — table shows Recipient, Subject, Sent time, Status (sent/failed). Includes loading state, empty state, search, refresh button.

Bull Board

Open: <http://localhost:4000/admin/queues>

Bull Board displays waiting, active, delayed, completed, and failed jobs. The route requires authentication.

Restart Persistence

PostgreSQL stores permanent email job state. Redis stores BullMQ delayed jobs. Docker volumes preserve infrastructure data.

If the backend restarts:

1. Future jobs remain in Redis.
2. EmailJob records remain in PostgreSQL.
3. The worker reconnects.
4. Already-sent jobs are skipped.
5. Future jobs continue.
6. Jobs are not recreated blindly.
7. Duplicate delivery is prevented.

Behavior with 1000+ Emails

For 1000 scheduled recipients:

1. Create 1000 EmailJob records.
2. Create 1000 delayed BullMQ jobs.
3. Store delayed jobs in Redis.
4. Process only the configured number concurrently.
5. Respect delay spacing.
6. Apply sender/hour rate limiting.
7. Reschedule excess jobs.
8. Keep every job separately searchable.

With `WORKER_CONCURRENCY=5`, only five jobs are processed simultaneously.

API Routes

Authentication

```
GET /api/auth/google
GET /api/auth/google/callback
GET /api/auth/me
POST /api/auth/logout
```

Campaigns

```
POST /api/campaigns
GET /api/campaigns
GET /api/campaigns/:id
```

Emails

```
GET /api/emails/scheduled
GET /api/emails/sent
GET /api/emails/search?q=
```

Senders

```
GET /api/senders
POST /api/senders
```

Slack

```
GET /api/slack/connect
GET /api/slack/callback
```

```
GET /api/slack/status
POST /api/slack/disconnect
```

Health

```
GET /health
```

Build and Test Commands

```
npm.cmd --prefix backend test
npm.cmd --prefix backend run build
npm.cmd --prefix frontend run build
docker compose ps
Invoke-RestMethod http://localhost:4000/health
```

Demo Procedure

1. Start Docker Desktop.
2. Confirm Engine running.
3. Run docker compose up -d.
4. Confirm PostgreSQL, Redis, and Elasticsearch are healthy.
5. Start the backend.
6. Start the BullMQ worker.
7. Start the frontend.
8. Open <http://localhost:5173>.
9. Login with Google.
10. Select the sender.
11. Click Compose New Email.
12. Enter subject and body.
13. Upload sample-leads.csv.
14. Choose a future start time.
15. Set delay to 10 seconds.
16. Set hourly limit to 100 for a simple test.
17. Click Schedule emails.
18. Open Scheduled Emails.
19. Open Bull Board.
20. Wait for the scheduled time.
21. Refresh the dashboard.
22. Open Sent Emails.
23. Check the real inbox if using Gmail, Brevo, SendGrid, Mailgun, or SES.
24. Open the Ethereum preview URL if using Ethereum.
25. Search for a recipient.
26. Restart the backend while future jobs exist.
27. Confirm future jobs remain.
28. Confirm sent jobs are not duplicated.

Troubleshooting

Docker not recognized:

```
$env:Path += ";C:\Program Files\Docker\Docker\resources\bin"
```

Node or npm not recognized:

```
$env:Path += ";C:\Program Files\nodejs"
```

PowerShell blocks npm: Use npm.cmd.

Docker daemon unavailable:

1. Open Docker Desktop.
2. Wait for Engine running.

3. Run `docker info`.
4. Confirm WSL2 and Ubuntu are installed.
5. Run `docker compose up -d`.

Google OAuth not configured:

- Check `GOOGLE_CLIENT_ID`.
- Check `GOOGLE_CLIENT_SECRET`.
- Check `GOOGLE_CALLBACK_URL`.
- Restart the backend.
- Confirm the redirect URI exactly matches Google Cloud.
- Add the Google account as a test user.

Gmail returns error 535:

- Use a Gmail App Password.
- Do not use the normal Gmail password.
- Do not use Google OAuth secrets as SMTP credentials.
- Confirm `SMTP_USER` is the same Gmail account.
- Remove spaces from the App Password.
- Restart backend and worker.
- Create a new campaign because failed jobs remain failed.

Dashboard shows SENT but Gmail receives nothing:

- Check the worker terminal.
- If an Ethereum preview URL appears, Ethereum is active.
- Ethereum does not deliver to Gmail.
- Use Gmail SMTP or another real SMTP provider for inbox delivery.

Jobs remain scheduled:

- Confirm the start time is in the future.
- Confirm the worker terminal is running.
- Confirm only one project worker is running initially.
- Check Bull Board.
- Check Redis and PostgreSQL containers.
- Refresh the dashboard.

Sharing the Project With Another Laptop

Share:

- `backend/`, `frontend/`
- `docker-compose.yml`
- `sample-leads.csv`
- `README.md`, `package.json`
- `.env.example` files
- Prisma migration files

Do not share:

- `backend/.env`, `frontend/.env`
- Google Client Secret
- Gmail App Password
- Ethereum password
- Slack access token
- Any production credentials

On the new laptop:

1. Install Node.js.
2. Install Docker Desktop.
3. Install WSL2/Ubuntu if Docker uses WSL2.
4. Copy the project folder.
5. Open PowerShell in the project folder.

6. Run `docker compose up -d`.
7. Run `npm.cmd install`.
8. Run `npm.cmd -prefix backend install`.
9. Run `npm.cmd -prefix frontend install`.
10. Copy `.env.example` files to `.env`.
11. Add new laptop credentials.
12. Run `Prisma generate`.
13. Run `Prisma migration or deploy`.
14. Start backend.
15. Start worker.
16. Start frontend.
17. Open `http://localhost:5173`.

Google OAuth callback URLs and Slack callback URLs must be configured for the new laptop.

Stop Commands

Stop backend, worker, and frontend using `Ctrl+C`.

Stop Docker containers (preserves volumes):

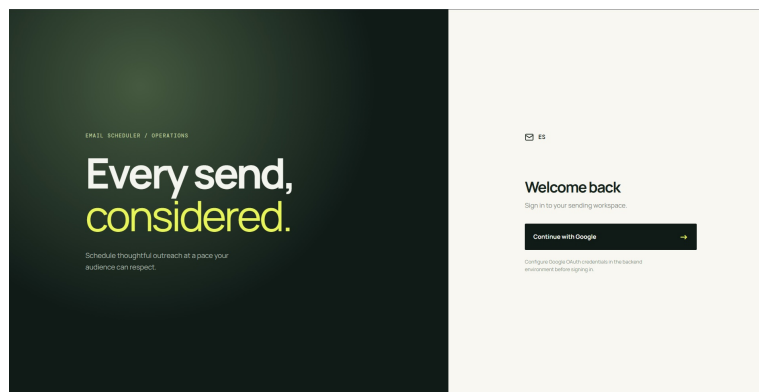
```
docker compose down
```

Delete all project data (use with caution):

```
docker compose down -v
```

Application Screenshots

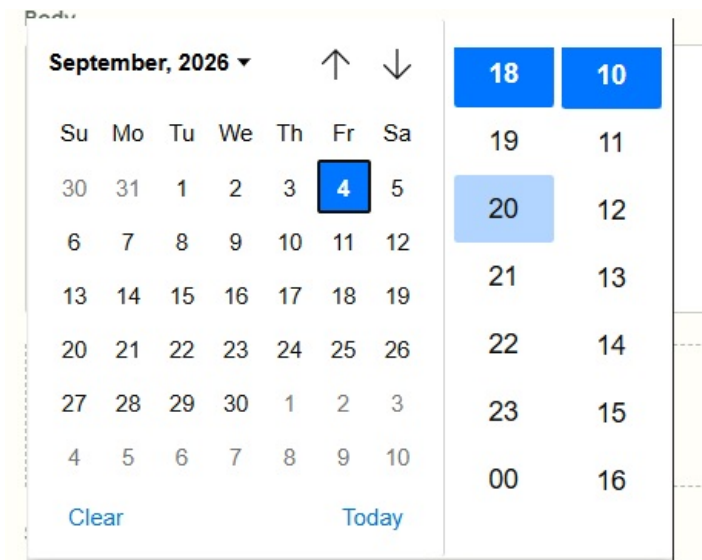
Login screen



Login screen

The split-screen login view: product messaging on the left, “Continue with Google” sign-in on the right.

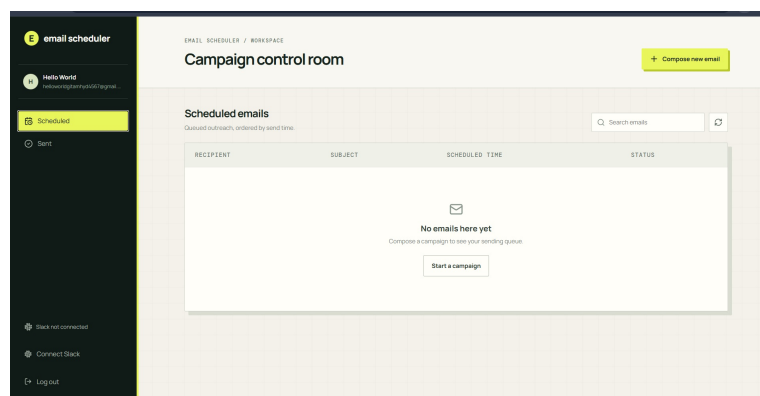
Schedule date/time picker



Schedule picker

The date and time picker used when composing a campaign, for choosing the future start time.

Campaign control room (dashboard)



Dashboard

The main workspace after login: Scheduled/Sent navigation, Slack connection status, and the scheduled emails table with search and refresh.

Final Explanation

The project is an email scheduler and delivery pipeline.

The frontend collects the campaign data. The backend validates and stores the campaign. PostgreSQL stores users and email jobs. BullMQ schedules one delayed job per recipient. Redis stores the queue and rate-limit counters. The worker claims and sends jobs. SMTP sends each email individually. Elasticsearch indexes emails for search. Bull Board monitors the queue. Slack reports rate-limit events. Idempotency prevents duplicate sends.

Docker provides PostgreSQL, Redis, and Elasticsearch. Ethereum is used for fake SMTP preview testing. A real SMTP provider is required when messages must arrive in real Gmail inboxes.

The complete email flow:

User
-> React dashboard

- > Express API
- > PostgreSQL campaign
- > One EmailJob per recipient
- > BullMQ delayed job
- > Redis queue
- > Email worker
- > Rate-limit check
- > SMTP delivery
- > PostgreSQL status update
- > Elasticsearch indexing
- > Sent Emails dashboard